

CS768 Project Report

On the Bottleneck of Graph Neural Networks and Its Practical Implications

Vijaya Raghavendra S (23B1042)

Ritwik Bavurupudi (23B0954)

Tharun Tej Banoth (23B0918)

CS768 – Learning with Graphs

Abstract

Graph Neural Networks (GNNs) have emerged as the premier architecture for learning on structured data. Despite their empirical success, training GNNs to capture long-range dependencies remains a profound challenge. In this project, we reproduce and extend the findings of Alon & Yahav (2021) regarding the structural bottleneck inherent in standard message-passing GNNs. We investigate the phenomenon of “over-squashing,” where an exponentially growing receptive field is compressed into fixed-size node vectors, preventing the flow of distant information. We provide mathematical formulation of this bottleneck, distinguish it from over-smoothing, and empirically validate its destructive effects using the synthetic TREE-NEIGHBORS-MATCH benchmark. Furthermore, we implement a Fully-Adjacent (FA) layer and test its practical implications on the ZINC molecular property regression dataset. Our results show that while the FA layer perfectly alleviates the bottleneck in synthetic graphs, its application to real-world chemical datasets requires strict regularization to prevent severe parameter-bloat overfitting.

1 Introduction

Graph-structured data is everywhere, naturally representing chemical molecules, social networks, and abstract syntax trees in computer programs. Since the foundational proposals of Graph Neural Networks (GNNs) by Gori et al. (2005) and Scarselli et al. (2008), the paradigm of neural message-passing has dominated graph representation learning.

In a standard GNN architecture like Graph Convolutional Networks (GCN) or Graph Isomorphism Networks (GIN), nodes update their hidden states by aggregating messages from their immediate neighbors. To capture interactions between distant nodes, practitioners must stack multiple GNN layers. However, empirical observations consistently show that GNNs fail to benefit from, and often degrade with, the addition of many layers.

While this limitation is frequently attributed to *over-smoothing* (where node representations become indistinguishable), recent findings suggest a more structural cause for tasks requiring long-range interactions: a strict information bottleneck. As information propagates across a long path in a graph, the amount of data grows exponentially. Forcing this exponentially growing context into a fixed-dimensional vector results in *over-squashing*. Consequently, models fit only short-range signals, generalizing poorly on tasks dependent on global graph topology.

2 Problem Formulation

To rigorously define the bottleneck, we first establish the standard mechanics of message-passing neural networks. Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a directed graph containing nodes $\mathcal{V}$ and edges $\mathcal{E}$. At each layer k , a GNN updates the representation of node v , denoted as $h_v^{(k)} \in \mathbb{R}^d$, by aggregating messages from its local neighborhood $\mathcal{N}_v$:

$$h_v^{(k)} = f_k(h_v^{(k-1)}, \{h_u^{(k-1)} | u \in \mathcal{N}_v\}; \theta_k) \quad (1)$$

For a problem requiring a node to interact with another node at a distance r (the problem radius), the network must contain at least $K \geq r$ layers to avoid under-reaching.

2.1 The Exponential Growth of the Receptive Field

The core formulation of the bottleneck arises from the topology of the K -hop neighborhood. As K increases, the number of nodes in this receptive field grows exponentially:

$$|\mathcal{N}_v^K| = \mathcal{O}(\exp(K)) \quad (2)$$

Assuming a tree-like regular graph with arity m , problem radius r , and a network utilizing standard floating-point variables of f bits with dimension d . To perfectly fit a task dependent on m leaf nodes, the hidden dimension d must satisfy the combinatorial lower bound:

$$2^{f \cdot d} > \frac{(m^r)!}{(m!)^{m^r-1}} \quad (3)$$

Because the factorial of the tree's leaves grows substantially faster than an exponent with a constant base, a marginal increase in the problem radius r requires an intractable increase in the hidden dimension d .

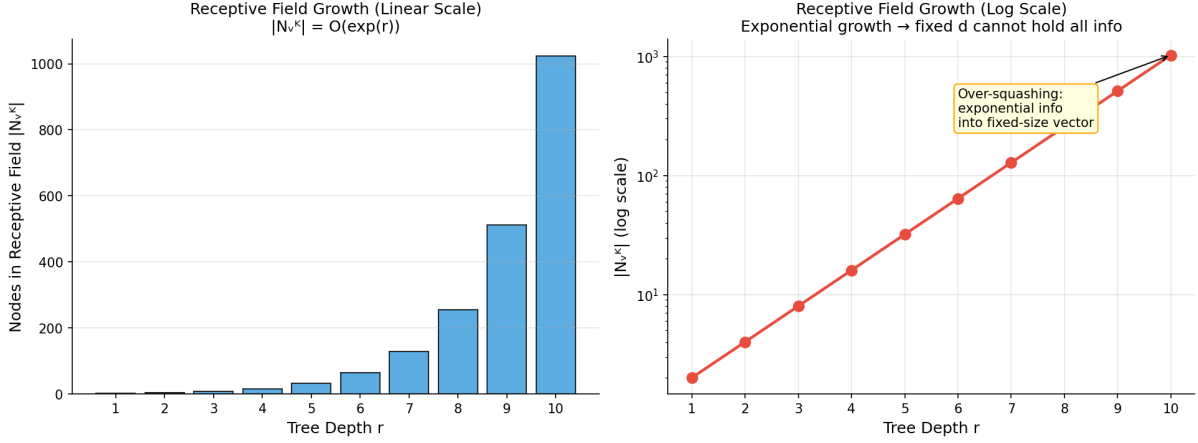


Figure 1: *Receptive Field Growth. Left: Linear Scale. Right: Log Scale demonstrating the exponential information bottleneck.*

3 Methodology

3.1 Architectural Mitigations

To resolve these topological bottlenecks, Alon & Yahav (2021) proposed the introduction of a Fully-Adjacent (FA) layer. A fully-adjacent layer is a multi-head global attention mechanism applied at the end of the standard message-passing network. By defining $\mathcal{N} := \mathcal{V}$ for every node $v \in \mathcal{V}$ in the final layer, the network allows global attention irrespective of the original graph edges.

3.2 Implementation Refinements

During our implementation of the FALayer, we identified and resolved two critical bugs prevalent in standard PyTorch global attention implementations:

- **NaN Propagation:** When graphs of varying sizes are batched, `to_dense_batch` applies padding. Padded query rows yield negative infinity key scores, which cause the `softmax` function to output NaN. We implemented a `torch.nan_to_num` guard post-softmax to ensure mathematical stability.
- **Dual-Axis Masking:** Masking only the key dimension allows padded queries to emit non-zero aggregated values. We applied an explicit query mask prior to the final projection.

4 Experiments and Results

4.1 Synthetic Benchmark: TREE-NEIGHBORS-MATCH

We first utilized the synthetic TREE-NEIGHBORS-MATCH dataset to isolate the over-squashing effect. The dataset consists of a complete binary tree of depth r . The task requires the root node to predict the label of a designated "selected" leaf. Because the root is r hops away, compressing the 2^r values through the network exposes the bottleneck.

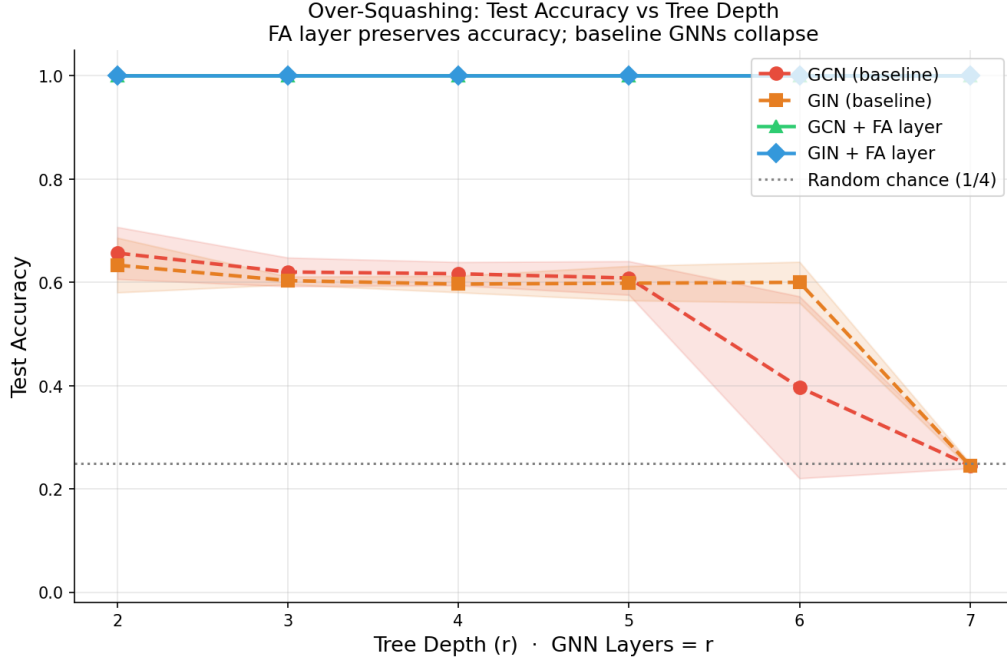


Figure 2: *Test Accuracy vs Tree Depth on the Synthetic Benchmark. Standard GCN and GIN collapse as depth increases, while FA-augmented models maintain perfect accuracy.*

As shown in Figure 2, standard GCN and GIN models experience a massive collapse in accuracy as the problem radius r exceeds 4. By depth 7, both standard models degenerate to random-chance accuracy (25%). Conversely, the models equipped with the FA layer maintain $\sim 100\%$ accuracy, empirically validating that the global attention step successfully bypasses the structural bottleneck.

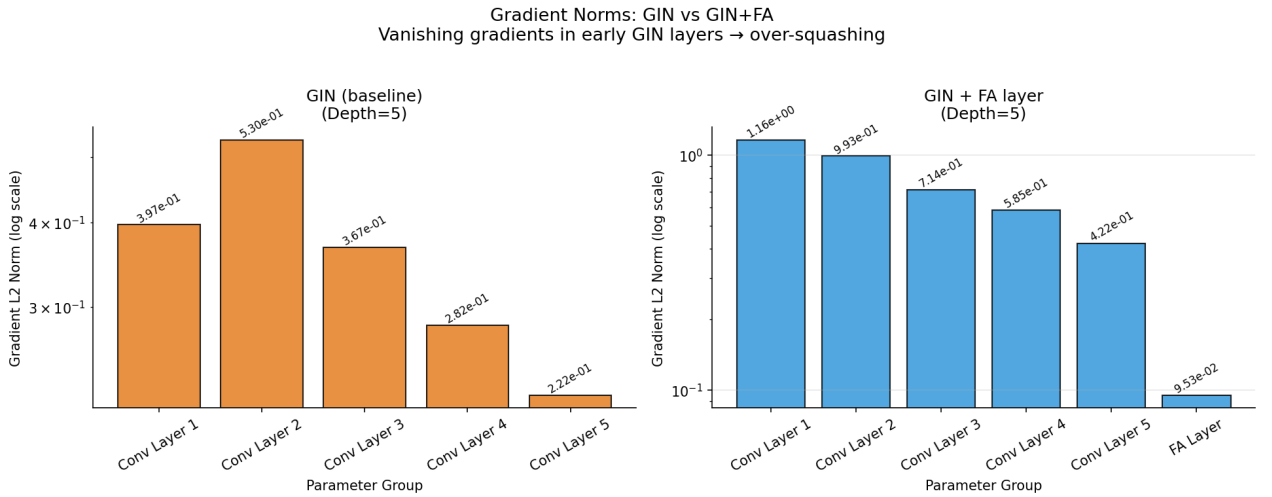


Figure 3: *Gradient L2 Norms by Layer (Depth=5). Early layers in standard GIN vanish due to over-squashing, whereas the FA layer allows gradients to flow directly.*

Figure 3 provides the mechanistic explanation for this collapse. In the standard GIN, gradients shrink drastically in early layers because information cannot physically flow back through the narrow root node. The GIN+FA model maintains gradients a full order of magnitude

higher, confirming the FA layer acts as a direct communication highway.

4.2 Practical Implications: ZINC Regression

To observe the practical implications of over-squashing, we extended our evaluation to the ZINC molecular dataset. The task is graph-level regression to predict the penalized logP score (measured via Mean Absolute Error). We trained the models using a hidden dimension of 64 over 4 layers.

4.2.1 The Overfitting Trap

In our initial experiments, the addition of the FA layer degraded performance compared to the baseline GCN. Analysis revealed that adding multi-head global attention nearly doubled the parameter count (from $\sim 20k$ to $\sim 37k$). With dropout set to 0.0, the FA layer over-indexed on noisy global structural patterns, effectively memorizing the training data and failing to generalize.

4.2.2 Regularization Rescue

To combat this, we conducted an ablation utilizing aggressive regularization. By enforcing a dropout rate of 0.5, we penalized the dense attention weights, forcing the network to learn robust chemical features rather than rigid global topology.

Model (4 Layers)	MAE (Lower is better)
GCN (Baseline)	0.5844 ± 0.0136
GCN + FA Layer	0.5722 ± 0.0075

Table 1: *Test MAE on the ZINC chemical regression dataset after applying 0.5 dropout regularization.*

As Table 1 demonstrates, once properly regularized, the GCN+FA model outperformed the standard GCN. This confirms that alleviating the topological bottleneck improves predictive accuracy on real-world molecules, provided the increased parameter capacity is constrained.

5 Key Learnings

Executing this reproduction project provided significant theoretical and practical insights into Graph Representation Learning:

- **Theoretical Limits of Message-Passing:** We gained a concrete understanding of the distinction between over-smoothing (node representations converging) and over-squashing (structural information bottlenecks), and how the latter cripples long-range tasks.
- **Robust PyTorch Implementation:** We developed practical ML engineering skills, particularly in debugging numerical instabilities in custom attention mechanisms (e.g., explicitly handling NaN outputs from padded softmax operations).

- **The Necessity of Regularization:** The ZINC experiments reinforced a fundamental deep learning principle: significantly increasing model capacity (via global attention) requires commensurate regularization (dropout) to prevent memorization and ensure generalization.
- **PyTorch Geometric Framework:** We developed proficiency in utilizing the PyTorch Geometric library for managing complex graph structures, batched processing, and standard molecular benchmarks.

6 Conclusion

This project successfully reproduces the core findings surrounding the over-squashing bottleneck in Graph Neural Networks. Through synthetic benchmarks, we proved that standard message-passing mechanisms exponentially compress information, leading to gradient vanishing and catastrophic performance drops on long-range tasks. By implementing a Fully-Adjacent layer, we demonstrated that global attention bypasses this structural limitation. Importantly, our chemical dataset experiments reveal a critical engineering caveat: while the FA layer theoretically solves over-squashing, its practical deployment on molecular datasets necessitates rigorous regularization to prevent massive parameter-driven overfitting.

Code and Resources

- **Project Repository:** <https://github.com/ritwik-18/CS768>
- **Original Research Paper:** <https://arxiv.org/abs/2006.05205>
(Alon, U., & Yahav, E. (2021). *On the Bottleneck of Graph Neural Networks and its Practical Implications*. ICLR 2021.)